CIT 5940 - Module 6 Programming Assignment
Binary Search Trees

Contents

Contents 1
Assignment Overview **2**
Learning Objectives **2**
Advice **2**
1 Setup **3**
2 Requirements **3**
2.1 Structure and Compiling 3
2.2 Functionality Specifications 4
2.2.1 General Requirements 4
2.2.2 Inverted Index 4
2.2.2.1 addDocument (int docID, String text) 4
2.2.2.2 search (String query) 6
2.2.2.3 removeDocument(int docID) 6
2.2.2.4 getIndex() 6
2.2.2.5 Main Functionality Overview 7
2.3 Writing Test Cases 8
3 Submission **9**
3.1 Pre-submission Check 9
3.2 Gradescope Submission 9
4 Grading **10**
4.1 Grading Overview 10
4.2 Rubric Items 10

Assignment Overview

In this module, you will build a free text search engine that will serve as a foundation for a search engine you will develop in the final project. In this homework assignment, you will be using an inverted index implemented with a Binary Search Tree (BST) where each node contains a keyword and a $Set < Integer >$ of document IDs representing all documents that contain that word. This assignment introduces concepts behind document indexing and retrieval, text normalization, and Boolean "AND" search functionality.

Learning Objectives

- Understand and implement Binary Search Trees (BSTs)
- Construct an inverted index for keyword-based document retrieval
- Apply text tokenization and normalization for search
- Normalize text for robust search

Advice

We highly encourage you to utilize the course discussion forum to discuss the project with classmates and TAs. Before asking the question, please search in the forum to see if there are any similar questions asked before. Please check *all* suggestions listed there.

For this and all other assignments in this course, we recommend that you work on your local computer using an integrated development environment (IDE) such as Eclipse, IntelliJ, Visual Studio Code, or whatever IDE you used for any prior Java courses that you are familiar with.
Although you will need to upload your solution to Gradescope for grading (see the Submission Section below) and can check your code using the autograder there, we recommend using industry standard IDE's such as Eclipse or IntelliJ so that you become more used to them and can take advantage of their features.

If you have trouble setting up your IDE or cannot get the starter code to compile, please post a note in the discussion forum and a member of the instruction staff will try to help you get started.

We expect that you will create your own JUnit tests to validate your solution before submission.

1 Setup

1. Begin by downloading the files `InvertedIndex.java`, `Main.java`, `InvertedIndexTest.java`, `all_articles_filtered_sample_10.csv` from Canvas. The main files you will work with are `InvertedIndex.java` and `Main.java`. These contain the unimplemented methods for the code that you will write in this assignment.
2. Before compiling, create a package named edu.upenn.cit5940 in your IDE and place the provided files inside (if using Eclipse, you can click the quick-fix "Move to package edu.upenn.cit5940" on the error). This will ensure your local environment is configured correctly.
3. Review the current function headers. You will want to understand the various functions and how they work together before making changes!

2 Requirements

2.1 Structure and Compiling

1. You SHOULD use a Java Development Kit (JDK) version of 25 or higher.
2. You MAY use a JDK version higher than 25, but you MUST set the Java language level to 25 for compatibility with Gradescope.
3. You MUST NOT change any of the method signatures for any of the provided methods. This includes the parameter lists, names, and return value types, etc.
4. You MUST leave the `InvertedIndex` and `Main` classes in package edu.upenn.cit5940.
5. You MUST NOT create additional `.java` files for your solution.
6. You SHOULD create JUnit tests to help validate your code. These tests SHOULD be a part of your solution. We do expect you to test your code thoroughly with JUnit tests before submission.
7. You MUST follow our directives in the starter code comments. This means if we ask you to not change a variable's value, you MUST NOT change that variable's value.
8. You MAY create additional helper functions, as long as they meet the other requirements of the assignment (e.g. does not crash with invalid inputs, etc.).
9. You MUST fill out the required Academic Integrity signature in the comment block at the top of your submission file.

2.2 Functionality Specifications

2.2.1 General Requirements

1. Your method MUST NOT crash if a user enters an invalid input. Remember you do not have control over what a user inputs as arguments. For example, all methods should handle `null` inputs gracefully or as specified.
2. You MUST NOT overload any of the provided methods.
3. You MAY have any `import` statements in your solution.
4. You MAY use methods of the elements, for example `equals` or `compareTo` from `java.lang.String`.
5. You MUST NOT throw any exceptions, either deliberately or by failing to handle them, unless otherwise specified in these instructions
6. You MAY create your own helper methods.

2.2.2 Inverted Index

The `Inverted Index` class uses a binary search tree for keyword-based document retrieval. It supports adding documents, querying them, and inspecting or removing indexed entries.

2.2.2.1 addDocument (int docID, String text)

You MUST implement the `addDocument` method as follows:

1. it MUST take the following parameters:
   (a) `int docID`: the ID of the document to be added.
   (b) `String text`: the text content of the document to be added.
2. It MUST split the input text into lowercase words with all punctuation removed.
3. It MUST add each word to a binary search tree (BST), where each node represents a keyword and stores a `Set <Integer>` of document IDs.
4. It MUST ignore empty strings.
5. It MUST skip any STOP words from the provided NLTK STOP WORDS list which also includes high frequency but low-information words. STOP words are common words (for example: “the”, “and”, “is”, “of”, “to”) that are extremely common and do not significantly help distinguish between documents.
6. If a keyword already exists in the BST, it MUST add the new document ID to the existing set for that keyword.
7. It MUST NOT store duplicate document IDs for a word.
8. It MUST use a helper method to tokenize the text. The tokenization MUST: convert text to lowercase,

preserve hyphens (-) when they appear between letters or digits, so “in-order” remains “in-order”, remove all other punctuation or symbols, replacing them with spaces. Split on one or more whitespace characters to produce tokens.

You can use the following regex statement:

```
// tokenize ensures the text is lowercase all non-alphanumeric (except spaces and hyphens)

// characters are replaced with a space and then split on

// whitespace

text = text.toLowerCase().replaceAll("[^a-z0-9\\s-]", " ");



// remove leading hyphens

text = text.replaceAll("^-+", "");

//remove trailing hyphens

text = text.replaceAll("-+$", "");



// remove hyphens after spaces

text = text.replaceAll("\\s-+", " ");

// remove hyphens before spaces

text = text.replaceAll("-+\\s", " ");
```

2.2.2.2 search (String query)

You MUST implement the `search(String query)` method as follows:

1. it MUST take the following parameters:
   (a) `String query`: the user query string
2. it MUST return a `Set <Integer>` containing all document IDs that contain all the words in the query.
   (a) Clarification: Multiple word queries use Boolean AND. If the query tokens are {java, python, …} the result should be an intersection of their document-ID sets. This would mean the more words there are fewer or equal results. If any token is not in the index you must return an empty set. The intent behind the regular vanilla version of this assignment is to demonstrate exact match Boolean retrieval: adding words narrows results through set intersection.
3. It MUST process the query string the same way as `addDocument`, including tokenization.
4. It MUST process the same stop word filtering used in `addDocument` when processing the query string, so that searches ignore stop words consistently.
5. If any of the words do not exist in the index, it MUST return an empty set.
6. If the query string is empty, it MUST return an empty set.
7. It MUST NOT modify the index.
8. If the index is empty (root == null), it MUST return an empty set not null

2.2.2.3 removeDocument(int docID)

You MUST implement the `removeDocument (int docID)` method as follows:

1. it MUST take the following parameters:
   (a) `int docID`: the ID of the document to be removed.
2. it MUST remove the document ID from every keyword node's document set in the BST.
3. It MUST NOT remove nodes from the tree, only the ID from the sets.
4. If the document ID is not present in a node's set, it MUST do nothing for that node.

2.2.2.4 getIndex()

You MUST implement the `getIndex()` method as follows:

1. It MUST NOT take any parameters.
2. it MUST return a `Map <String, Set <Integer>>`, where:

(a) The key is the keyword.

(b) The value is the set of document IDs in which the keyword appears.

3. It MUST perform an in-order traversal to ensure that the map is sorted alphabetically by keyword.

   (a) Option 1: Perform an in-order traversal of the BST and construct the returned map in the same order as the traversal

   (b) Option 2: Use a TreeMap to automatically order keys alphabetically. If you prefer this approach then an in-order traversal is not required.

4. It MUST NOT modify the internal BST.

5. If the tree's root is null you MUST return an empty map, not null

2.2.2.5 Main Functionality Overview

1. Load article data (title and body) from a CSV file, assigning each article a unique document ID (starting from 1).

   The following library can help complete this requirement:

   (a) Using OpenCSV

   - This requires downloading the opencsv-5.12.jar (download the latest version) from the following website Maven Repository
   - Place the JAR in the project directory either at the root or at the `lib/` folder (which you may need to create)
   - Include the following imports at the top of the file:
     ```
     import com.opencsv.CSVReader;
     import com.opencsv.CSVReaderBuilder;
     ```
   - You will also need to import commons-beanutils, commons-collections4, commons-lang3, and commons-text as co-compilation dependencies. For more information, please see https://opencsv.sourceforge.net/dependencies.html. Please download the .jar files from say Maven Repository and import as library in your local project accordingly.

2. Store articles in a `Map<Integer, String>` for retrieval.

3. Add each article to the InvertedIndex using the addDocument(int docID, String text) method.

   Clarifications:
   What does "article" mean and how to build combinedText:

   Article text format for this assignment: For each CSV row, read the title and body, then build a single string that looks like this combinedText = title + "\n" + body. Store this in Map<Integer,

String> docTexts, and also pass the **same** combinedText to index.addDocument(docID, combinedText).

When and why to use helper methods:

extractTitle(combinedText) returns the first line (the title). While extractFirstSentence(combinedText) takes everything after the first newline as the body, finds the first sentence, and prints at most 200 characters with an ellipsis (...) if needed.

These helpers are used only when printing results. They do not affect tokenization or indexing.

What is the 200-character cutoff:

This is the difference between indexing vs display: Indexing uses the full combined text (title + full body). The 200-character limit applies only to the preview printed to the terminal.

Which CSV fields to read:

The dataset includes title and body columns. Use OpenCSV to read those two fields for each row. Skip rows where both fields are blank.

4. Steps 5 to 9 have already been completed for you and are listed here for your reference:
5. Prompt the user to enter queries interactively via the terminal.
6. Use search (String query) to retrieve and print matching document IDs and their content.
7. Blank or malformed lines in the CSV are skipped.
8. If the search yields no results, the program prints a message such as No matches found!
9. The program continues until the user types exit.

2.3 Writing Test Cases

Upon completion of each functional requirement, always conduct thorough testing of the corresponding functions. Consult the guidelines on composing JUnit tests and creating test cases in Module 1. You MUST develop JUnit tests to cover all corner cases you can think of for each of your implemented functions in order to get full credit. This testing component carries a weight of approximately 26% towards the total assignment score in this assignment. Please note that you do not need test cases for helper functions.

3 Submission

3.1 Pre-submission Check

Before you submit, please double check that you followed all the instructions, especially the items in the Structure and Compiling section above.

3.2 Gradescope Submission

You will be expected to submit:

1. When you are ready to submit the assignment, go to the Module 6 Programming Assignment Submission item and click the button to go to the Gradescope platform.
2. Once you have logged into Gradescope, locate the assignment name you would like to submit. This should be the first thing that appears in the Dashboard..
3. Upload your solution and any JUnit test files via file upload or a private GitHub repo. You will need to submit the following files:

   (a) InvertedIndex.java

   (b) Main.java

   (c) InvertedIndexTest.java
4. When you are sure you are ready to submit, confirm at the prompt.
5. You will see quite a bit of output after a while (which can be at most a few minutes), even if all the tests pass. At the bottom of the output, starting at “Autograder Results” you will see the number of successful and failed test cases.
6. If you want to update your code and resubmit, for example if you did not pass all of the tests, simply edit your code and resubmit.

4 Grading

4.1 Grading Overview

1. There are no hidden tests. You will be able to see the name of each test, if it passed or failed, and the associated point value.
2. Failed tests will have brief feedback on the specifics of the failure. We will not provide the exact test cases.
3. You have unlimited submissions, until the due date of the assignment as noted in the syllabus (or your approved extension).
4. The score on the due date is final. Scores will sync to Canvas relatively immediately.

4.2 Rubric Items

This assignment will be marked on 6 rubric items.

- **Inverted Index - addDocument** : 15 points
- **Inverted Index - search** : 12 points
- **Inverted Index - removeDocument** : 15 points
- **Inverted Index - getIndex** : 9 points
- **A `main` method in `Main.java`** : 15 points
- **JUnit Test Cases** : 18 points

Total : 69 points from autograding (+15 points manual grading for the `Main.main()` method).

Good Luck!